DATE: 17/09/2025

FAMILIARIZATION: 1

FAMILIARISE GCC

AIM:

Advanced use of gcc : Important Options -o, -c, -D, -l, -I, -g, -O, -save-temps, -pg

DESCRIPTION:

- gcc stands for GNU C/C++ Compiler
- A popular console-based compiler for Unix/Linux based platforms andothers; can cross-compile code for various architectures gcc performs all of these:
  - preprocessing
  - compilation
  - assembly and linking

Options: -c

- gcc performs compilation and assembly of the source file without linking.
- The output are usually object code files; they can later be linked and form the desired executables.
- Generates one object file per source file keeping the same prefix (before) of the filename.

Options: -D

- gcc defines a macro to be used by preprocessor.

Options: -o

- Places resulting file into the filename specified instead of the default one.
- Can be used with any generated files (object, executables, assembly, etc.)
- If you have the file called source.c; the defaults are: –source.o if -c was specified – a.out if executable.
- These can be overridden with the -o option.

Options: -g

- Includes debugging information in the generated object code. This information can later be used in gdb.
- gcc allows to use -g with the optimization turned on (-O) in case there is a need to debug or trace the optimized code.

Options: -ggdb

- In addition to -g produces the most GDB friendly output if enabled.

Options: -O, -O1, -O2, -O3, -O0, -Os

- Various levels of optimization of the code.
- -O1 to -O3 are various degrees of optimization targeted for speed
- If -O is added, then the code size is considered
- -O0 means “no optimization”
- -Os targets generated code size (forces not to use optimizations resulting in bigger code).

Options: -I

- gcc to look for include files (.h).
- Can be any number of these.
- Usually needed when including headers from various-depth directories in non-standard places without necessity specifying these directories with the .c files themselves

Options: -pg

- gcc performs to compile a source file for profiling,specify the pg option when run the compiler.

Options: -save-temps

- gcc stores the normally temporary intermediate files permanently.

RESULT:

Familiarised gcc.

DATE: 17/09/2025

FAMILIARIZATION: 2

FAMILIARISE GDB

AIM:

Familiarisation with gdb :

Important Commands -break, run, next, print, display, help

DESCRIPTION:

gdb is the acronym for GNU Debugger. This tool helps to debug the programs written in C, C++, Ada, Fortran, etc. The console can be opened using the gdb command on terminal.

Commands

- run [args]: This command runs the current executable file.eg: The program was executed twice, one with the command line argument 10 and another with the command line argument, and their corresponding outputs were printed.
- break: The command break [function name] helps to pause the program during execution when it starts to execute the function. It helps to debug the program at that point.
- next or n: This command helps to execute the next instruction after it encounters the break point. Whenever it encounters the above command, it executes the next instruction of the executable by printing the line in execution.
- display: These command enables automatic displaying of expressions each time whenever the execution encounters a break point or the n command. The undisplay command is used to remove display expressions.
- print: This command prints the value of a given expression. The display command prints all the previously displayed values whenever it encounters a break point or the next command, whereas the print command saves all the previously displayed values and prints whenever it is called print[Expression].
- help: It launches the manual of gdb along with all list of classes of individual commands.

RESULT:

Familiarised gdb.

DATE: 17/09/2025

FAMILIARIZATION: 3

FAMILIARISE GPROF

AIM:

Using gprof : Compile, Execute and Profile

DESCRIPTION:

Compile

- The first step in generating profile information for your program is to compile and link it with profiling enabled.
- To compile a source file for profiling, specify the -pgoption when run the compiler.
- To link the program for profiling, if you use a compiler such as cc to do the linking, simply specify -pg in addition to usual options. The same option, -pg, alters either compilation or linking to do what is necessary for profiling.
- display: These command enables automatic displaying of expressions each time whenever the execution encounters a break point or the n command. The undisplay command is used to remove display expressions.

Execute

- Once the program is compiled for profiling, you must run it in order to generate the information that gprof needs. Simply run the program as usual, using the normal arguments, file names, etc. The program should run normally, producing the same output as usual. It will, however, run somewhat slower than normal because of the time spent collecting and writing the profile data.
- The way to run the program—the arguments and input that you give it—may have a dramatic effect on what the profile information shows. The profile data will describe the parts of the program that were activated for the particular input you use. For example, if the first command you give to your program is to quit, the profile data will show the time used in initialization and in cleanup, but not much else.

Profile

- Describes the GNU profiler,gpof, and how can use it to determine which parts of a program are taking most of the execution time. We assume that to know how to write, compile, and execute

RESULT:

Familiarised gprof.

ALGORITHM:

1. Start
2. Read the limit n.
3. Initialize i = 0, sum = 0.
4. Check whether i < n
    - If true, go to step 5
    - Else go to step 7
5. Read array element a[i].
6. Increment i by 1 and go back to step 4.
7. Set i = 0 again.
8. Check whether i < n
    - If true, go to step 9
    - Else go to step 11
9. Add a[i] to sum.
10. Increment i by 1 and return to step 8.
11. Display the sum.
12. Stop

OUTPUT:

Enter the limit:5

Enter the element:5

4

2

1

3

SUM = 15

DATE: 17/09/2025

EXPERIMENT: 4.1

SIMPLE ARRAY OPERATIONS -SUM OF ELEMENTS IN AN ARRAY

AIM:

Write a program to find sum of elements in an array

SOURCE CODE:

```
#include<stdio.h>
void main()
{
	int a[15], i=0, n, sum=0;
	printf("Enter the limit:");
	scanf("%d", &n);
	printf("Enter the element:");
	for(i=0; i<n; i++)
	{
		scanf("%d", &a[i]);
	}
	for(i=0; i<n; i++)
	{
		sum = sum + a[i];
	}
	printf("SUM = %d", sum);
}
```

RESULT:

Program is executed successfully and output is obtained.

ALGORITHM:

1. Start
2. Read the limit n of the array.
3. Initialize i = 0 and flag = 0.
4. Loop from i = 0 to i < n
    - Read a[i].
    - Increment i
5. Read the key element to search.
6. Loop again from i = 0 to i < n
    - If a[i] == key,
        - set flag = 1
        - break the loop.
7. If flag == 1, display "Search successful".
8. Else display "Search unsuccessful".
9. Stop

OUTPUT:

Enter the limit of array :4

Enter the array elements :44

55

22

33

Enter the key to be searched :33

Search successful, Element found at position: 4

DATE: 17/09/2025

EXPERIMENT: 4.2

SIMPLE ARRAY OPERATIONS –LINEAR SEARCH

AIM:

Write a program to perform linear search.

SOURCE CODE:

```
#include<stdio.h>
void main()
{
	int a[50], i, n, key, flag = 0, pos = -1;
	printf("Enter the limit of array :");
	scanf("%d", &n);
	printf("Enter the array elements :");
	for(i = 0; i < n; i++)
	{
		scanf("%d", &a[i]);
	}
	printf("Enter the key to be searched :");
	scanf("%d", &key);
	for(i = 0; i < n; i++)
	{
		if(a[i] == key)
		{
			flag = 1;
			pos = i;
			break;
		}
	}
	if(flag == 1)
	{
		printf("Search successful, Element found at position: %d", pos + 1);
	}
	else
	{
		printf("Search Unsuccessful, Element Not Found");
	}
}
```

RESULT:

Program is executed successfully and output is obtained.

ALGORITHM:

1. Start
2. Declare arrays:
    - a[50] – first sorted array
    - b[50] – second sorted array
    - c[100] – merged array
3. Read the size of first array → m.
4. Read the size of second array → n.
5. Read m sorted elements into array a.
6. Read n sorted elements into array b.
7. Initialize i = 0, j = 0, k = 0.
8. While i < m AND j < n:
    - If a[i] < b[j]
      → set c[k] = a[i], increment i.
    - Else
      → set c[k] = b[j], increment j.
    - Increment k.
9. Copy remaining elements of b if any (j < n).
10. Copy remaining elements of a if any (i < m).
11. Display merged array c.
12. Stop

OUTPUT:

Enter the size of the first array:4

Enter the Elements of first array (sorted order): 1

2

3

4

Enter the size of the second array:3

Enter the Elements of second array (sorted order): 6

7

8

Merged array = 1 2 3 4 6 7 8

DATE: 24/09/2025

EXPERIMENT: 5.1

SORTING ALGORITHMS -MERGE TWO SORTED ARRAYS

AIM:

Write a program to merge two sorted arrays.

SOURCE CODE:

```
#include<stdio.h>
void main()
{
	int a[50], b[50], c[100], m, n, i, j, k;
	printf("\n Enter the size of the first array:");
	scanf("%d", &m);
	printf("\n Enter the Elements of first array (sorted order): ");
	for(i = 0; i < m; i++)
	{
		scanf("%d", &a[i]);
	}
	printf("\n Enter the size of the second array:");
	scanf("%d", &n);
	printf("\n Enter the Elements of second array (sorted order): ");
	for(i = 0; i < n; i++)
	{
		scanf("%d", &b[i]);
	}
	i = 0;
	j = 0;
	k = 0;
	while(i < m && j < n)
	{
		if(a[i] < b[j])
		{
			c[k] = a[i];
			i++;
		}
		else
		{
			c[k] = b[j];
			j++;
		}
		k++;
```

```
        }
        while(i < m)
        {
                c[k] = a[i];
                i++;
                k++;
        }
        while(j < n)
        {
                c[k] = b[j];
                j++;
                k++;
        }
        printf("\n Merged array = ");
        for(i = 0; i < m + n; i++)
        {
                printf("%d ", c[i]);
        }
}
```

RESULT:

Program is executed successfully and output is obtained.

ALGORITHM:

1. Start
2. Declare an integer array arr[50], and variables n, i, j, temp.
3. Read the limit n.
4. Read n elements into the array.
5. Set i = 0.
6. Repeat the steps for i = 0 to i < n-1:
    - Set j = 0.
    - Repeat for j = 0 to j < n - 1 - i:
        - If arr[j] > arr[j+1], then
          → Swap arr[j] and arr[j+1].
7. Print the sorted array.
8. Stop

OUTPUT:

Enter the limit of array: 5

Enter the elements of the array:21

87

41

12

2

Sorted Array = 2 12 21 41 87

DATE: 24/09/2025

EXPERIMENT: 5.2

SORTING ALGORITHMS -BUBBLE SORT

AIM:

Write a program to find sum of elements in an array

SOURCE CODE:

```
#include<stdio.h>
void main()
{
	int n, i, j, array[50], temp;
	printf("Enter the limit of array: ");
	scanf("%d", &n);
	printf("\n Enter the elements of the array:");
	for(i = 0; i < n; i++)
	{
		scanf("%d", &array[i]);
	}
	for(i = 0; i < n - 1; i++)
	{
		for(j = 0; j < (n - i - 1); j++)
		{
			if(array[j] > array[j + 1])
			{
				temp = array[j];
				array[j] = array[j + 1];
				array[j + 1] = temp;
			}
		}
	}
	printf("\n Sorted Array = ");
	for(i = 0; i < n; i++)
	{
		printf("%d ", array[i]);
	}
}
```

RESULT:

Program is executed successfully and output is obtained.

ALGORITHM:

push()

1. Start
2. Check if top == MAX - 1
    - If true, print "Stack Overflow" and return
3. Else increment top
4. Insert the item at stack[top]
5. Display success message
6. Stop

pop()

1. Start
2. Check if top == -1
    - If true, print "Stack Underflow" and return
3. Else print element at stack[top]
4. Decrement top
5. Stop

peek()

1. Start
2. If top == -1
    - Print "Stack is Empty" and return
3. Else display stack[top]
4. Stop

display()

1. Start
2. If top == -1
    - Print "Stack is Empty" and return
3. Else print elements from top down to 0
4. Stop

DATE: 01/10/2025

EXPERIMENT: 6

STACK IMPLEMENTATION USING ARRAY

AIM:

Write a program to implement stack operations.

SOURCE CODE:

```
#include <stdio.h>
#define MAX 10
int stack[MAX];
int top = -1;

void push(int item) {
        if (top == MAX - 1) {
                printf("\n Stack Overflow!");
                return;
        }
        stack[++top] = item;
        printf("\n %d pushed to stack.", item);
}

void pop() {
        if (top == -1) {
                printf("\n Stack Underflow!");
                return;
        }
        printf("\n %d popped from stack.", stack[top--]);
}

void peek() {
        if (top == -1) {
                printf("\n Stack is Empty!");
                return;
        }
        printf("\n Top element is %d.", stack[top]);
}

void display() {
        if (top == -1) {
                printf("\n Stack is Empty!");
                return;
        }
```

main()

1. Start
2. Display menu
3. Read user choice
4. If
    - 1 → call push()
    - 2 → call pop()
    - 3 → call peek()
    - 4 → call display()
    - 5 → exit
5. Repeat until user chooses exit
6. Stop

OUTPUT:

```
Stack Operations Menu:
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter value to push: 12
12 pushed to stack.
Enter your choice: 1
Enter value to push: 23
23 pushed to stack.
Enter your choice: 1
Enter value to push: 34
34 pushed to stack.
Enter your choice: 3
Top element is 34.
Enter your choice: 4
Stack elements are: 34 23 12
Enter your choice: 2
34 popped from stack.
Enter your choice: 4
Stack elements are: 23 12
Enter your choice: 5
Exiting program.
```

```
        printf("\nStack elements are:");
        for (int i = top; i >= 0; i--) {
            printf(" %d", stack[i]);
        }
    }

    int main() {
    int choice, value;

    while (1) {
        printf("\n\n Stack Operations Menu:");
        printf("\n1. Push");
        printf("\n2. Pop");
        printf("\n3. Peek");
        printf("\n4. Display");
        printf("\n5. Exit");
        printf("\n Enter your choice: ");
        scanf("%d", &choice);
        switch (choice) {
            case 1:
                printf("\n Enter value to push: ");
                scanf("%d", &value);
                push(value);
                break;
            case 2:
                pop();
                break;
            case 3:
                peek();
                break;
            case 4:
                display();
                break;
            case 5:
                printf("\nExiting program.");
                return 0;
            default:
                printf("\nInvalid Choice!");
        }
    }
}
```

RESULT:

Program is executed successfully and output is obtained.

ALGORITHM:

isFull()

1. Start
2. If (rear+1) % MAX == front
   → Queue is full
3. Else queue is not full
4. Stop

isEmpty()

1. Start
2. If front == -1 && rear == -1
   → Queue is empty
3. Else queue is not empty
4. Stop

enqueue()

1. Start
2. Check isFull
   - If full → display message & return
3. Read element x
4. If queue is empty
   - Set front = rear = 0
5. Else
   - Set rear = (rear + 1) % MAX
6. Insert element at queue[rear]
7. Display success message
8. Stop

dequeue()

1. Start
2. Check isEmpty
   - If empty → display & return
3. Display deleted element
4. If only one element
   - Set front = rear = -1
5. Else
   - Update front = (front + 1) % MAX
6. Stop

DATE: 01/10/2025

EXPERIMENT: 7

CIRCULAR QUEUE USING ARRAY

AIM:

Write a program to implement circular queue.

SOURCE CODE:

```
#include<stdio.h>
#define MAX 5
int queue[MAX];
int front = -1, rear = -1;

int isFull() {
        if ((rear + 1) % MAX == front) {
                return 1;
        }
return 0;
}

int isEmpty() {
        if (front == -1 && rear == -1) {
                return 1;
        }
return 0;
}

void display() {
        int i;
        if (isEmpty()) {
                printf("\n QUEUE IS EMPTY \n");
                return;
        }
        printf("\n QUEUE ELEMENTS: ");
        i = front;
        do {
                printf("%d ", queue[i]);
                i = (i + 1) % MAX;
        } while (i != (rear + 1) % MAX);
}

void dequeue() {
        if (isEmpty()) {
        printf("\n QUEUE IS EMPTY \n");
        return;
```

search()

1. Start
2. Check isEmpty
3. Read key
4. Loop from front to rear circularly
5. If found → display position
6. If not found → print "not found"
7. Stop

main()

1. Start
2. Display menu repeatedly
3. Based on choice, call:
    - enqueue
    - dequeue
    - display
    - search
4. Exit if choice = 5
5. Stop

OUTPUT:

```
CIRCULAR QUEUE USING ARRAY
1. ENQUEUE
2. DEQUEUE
3. DISPLAY
4. SEARCH
5. EXIT
Enter your choice: 1
Enter the element to insert: 12
ELEMENT 12 INSERTED SUCCESSFULLY
Enter your choice: 1
Enter the element to insert: 23
ELEMENT 23 INSERTED SUCCESSFULLY
Enter your choice: 1
Enter the element to insert: 65
ELEMENT 65 INSERTED SUCCESSFULLY
```

```
        }
        printf("\n %d is DELETED \n", queue[front]);
        if (front == rear) {
                front = rear = -1;
        }
        else {
                front = (front + 1) % MAX;
        }
}

void enqueue() {
        int x;
        if (isFull()) {
                printf("\n QUEUE IS FULL \n");
                return;
        }
        printf("Enter the element to insert: ");
        scanf("%d", &x);
        if (isEmpty()) {
                front = rear = 0;
        }
        else {
                rear = (rear + 1) % MAX;
        }
        queue[rear] = x;
        printf("\n ELEMENT %d INSERTED SUCCESSFULLY \n", queue[rear]);
}

void search() {
        int key, i, found = 0;
        if (isEmpty()) {
                printf("\n QUEUE IS EMPTY \n");
                return;
        }
        printf("\nEnter the element to search: ");
        scanf("%d", &key);
        i = front;
        do {
                if (queue[i] == key) {
                        printf("\nElement %d found at position %d.\n", key, i);
                        found = 1;
                        break;
                }
                i = (i + 1) % MAX;
        } while (i != (rear + 1) % MAX);
```

Enter your choice: 3

QUEUE ELEMENTS: 12 23 65

Enter your choice: 4

Enter the element to search: 65

Element 65 found at position 2.

Enter your choice: 2

12 is DELETED

Enter your choice: 5

Exiting....

```
        if (!found) {
            printf("\nElement %d not found in the queue.\n", key);
        }
}

int main() {
int choice;
printf("\n CIRCULAR QUEUE USING ARRAY \n");

do {
        printf("\n1. ENQUEUE \n2. DEQUEUE \n3. DISPLAY \n4. SEARCH \n5. EXIT \n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1: enqueue();
                    break;
            case 2: dequeue();
                    break;
            case 3: display();
                    break;
            case 4: search();
                    break;
            case 5: printf("\n Exiting.... \n");
                    return 0;
            default: printf("\n INVALID CHOICE \n");
          }
} while (choice != 5);
return 0;
}
```

RESULT:

Program is executed successfully and output is obtained.

ALGORITHM:

insertFirst()

1. Start
2. Create newnode and allocate memory for newnode using malloc
3. Read the element to be inserted and set newnode→link = NULL
4. Check whether head == NULL
    - If true go to step 5
    - Otherwise go to step 6
5. Set head = newnode
6. Set newnode→link = head and head = newnode
7. Display the inserted element
8. Stop

insertLast()

1. Start
2. Create newnode and allocate memory for newnode using malloc
3. Set newnode→link = NULL and temp = head
4. Check whether head == NULL
    - If true go to step 5
    - Otherwise go to step 6
5. Set head = newnode
6. While temp→link != NULL is true go to step 7 otherwise go to step 8
7. Set temp = temp→link
8. Read the element to be inserted
9. Set temp→link = newnode
10. Display newly inserted element
11. Stop

insertLocation()

1. Start
2. Create newnode and allocate memory for newnode using malloc
3. Set newnode→link = NULL
4. Read the value of the node after which the newnode should be inserted and store in variable ‘key’
5. Set temp = head
6. While temp != NULL and temp→data != key is true go to step 7 otherwise go to step 8

DATE: 08/10/2025

EXPERIMENT: 8

SINGLY LINKED LIST

AIM:

Write a C program to implement singly linked list and it's operations.

SOURCE CODE:

```
#include<stdio.h>
#include<stdlib.h>
struct node {
                int data;
                struct node *link;
};
struct node *head = NULL;

void insertFirst() {
   struct node *newnode;
   newnode = (struct node*) malloc(sizeof(struct node));

   if (newnode == NULL) {
      printf("\n NO space available \n");
      return;
   }
   newnode->link = NULL;
   printf("\n Enter the value to insert to Front \n");
   scanf("%d", &newnode->data);
   if (head == NULL) {
      head = newnode;
   } else {
      newnode->link = head;
      head = newnode;
   }
   printf("\n Element inserted %d", newnode->data);
}

void insertLast() {
   struct node *temp = head, *newnode;
   newnode = (struct node*) malloc(sizeof(struct node));

   if (newnode == NULL) {
      printf("\n No space Available\n");
      return;
   }
   newnode->link = NULL;
```

7. Set temp = temp→link
8. Check whether temp == NULL is true go to step 9 otherwise go to step 10
9. Display the key value does not exist and return
10. Read the element to be inserted
11. Set newnode→link = temp→link and temp→link = newnode
12. Display the inserted element
13. Stop

deleteFirst()

1. Start
2. Check whether head == NULL is true go to step 3 otherwise go to step 4
3. Display "list is empty" and return
4. Set head = temp
5. Set head = temp→link
6. Display deleted element
7. Free temp
8. Stop

deleteLast()

1. Start
2. Check whether head == NULL
    - If true go to step 3 otherwise step 4
3. Display list is empty and return
4. Set temp = head and prev = NULL
5. Check whether temp→link == NULL
    - If true go to step 6 otherwise step 7
6. Free temp and set head = NULL and return
7. While temp→link != NULL is true go to step 8 otherwise go to step 9
8. Set prev = temp and temp = temp→link
9. Display the deleted element
10. Free temp and set prev→link = NULL
11. Stop

deleteLocation()

1. Start
2. Set temp = head and prev = NULL
3. Read the value of the node to be deleted in variable ‘key’
4. Check whether head == NULL
    - If true display list is empty and return
5. Check whether temp→data == key is true go to step 6 otherwise go to step 8
6. Set head = temp→link
7. Free temp and return

```
        printf("\n Enter the element to insert Last \n ");
        scanf("%d", &newnode->data);
        if (head == NULL) {
                head = newnode;
        } else {
                while (temp->link != NULL) {
                temp = temp->link;
        }
        temp->link = newnode;
        }
   printf("element inserted successfully %d", newnode->data);
}

void insertLocation() {
   int key;
   struct node *temp = head, *newnode;
   newnode = (struct node*) malloc(sizeof(struct node));
   if (newnode == NULL) {
      printf("\n No space available \n");
      return;
   }
   newnode->link = NULL;
   if (head == NULL) {
      printf("\n LIST empty \n");
      return;
   }
   printf("\n Enter the key were after you want to add Element \n");
   scanf("%d", &key);
   while (temp != NULL && temp->data != key) {
      temp = temp->link;
   }
   if (temp == NULL) {
      printf("\n Value Not Exist\n");
      return;
   }
   printf("\n Enter the Element to inserted\n");
   scanf("%d", &newnode->data);
   newnode->link = temp->link;
   temp->link = newnode;
   printf("value inserted successfully %d", newnode->data);
}

void deleteFirst() {
        struct node *temp = head;
        if (head == NULL) {
                printf("\n List Empty \n");
                return;
        }
        head = temp->link;
printf("\n Value deleted %d \n", temp->data);
free(temp);
}
```

search()

1. Start
2. Set temp = head, pos = 0 and found = 0
3. Check whether head == NULL
    - If true display list is empty and return
4. Read the element to be searched in 'val'
5. While temp != NULL is true go to step 6 otherwise go to step 10
6. Check whether temp→data == val
    - If true go to step 7 otherwise step 8
7. Display the position of val and set found = 1
8. Increment pos by 1
9. Set temp = temp→link
10. If !found is true display "value does not exist"
11. Stop

display()

1. Start
2. Set temp = head
3. Check whether temp == NULL is true display list is empty and return
4. While temp != NULL is true go to step 5 otherwise step 7
5. Display temp→data
6. Set temp = temp→link
7. Stop

main()

1. Start
2. Perform the given operations until choice != 9 using do-while loop
3. Read choice
4. Check the value of choice
5. When choice == 1
    - Call insertFirst()
    - Break
6. When choice == 2
    - Call insertLast()
    - Break
7. When choice == 3
    - Call insertLocation()
    - Break
8. When choice == 4
    - Call deleteFirst()
    - Break
9. When choice == 5
    - Call deleteLast()
    - Break

```
void deleteLast() {
        struct node *temp = head, *prev = NULL;
        if (head == NULL) {
                printf("\n Empty list \n");
                return;
    }
    if (temp->link == NULL) {
        printf("\n value %d deleted \n", temp->data);
        head = NULL;
        free(temp);
        return;
    }
    while (temp->link != NULL) {
        prev = temp;
        temp = temp->link;
    }
    printf("\nvalue %d deleted\n", temp->data);
    prev->link = NULL;
    free(temp);
}
void deleteLocation() {
        int key;
        struct node *temp = head, *prev = NULL;
        if (head == NULL) {
                 printf("\n Empty list \n");
                return;
        }
        printf("\n Enter the key that you want to delete\n");
        scanf("%d", &key);
        if (temp->data == key) {
                head = temp->link;
                printf("\n value %d is deleted \n", temp->data);
                free(temp);
                return;
        }
        while (temp != NULL && temp->data != key) {
                prev = temp;
                temp = temp->link;
        }
        if (temp == NULL) {
                printf("\n Value Not Exist\n");
                return;
        }
        prev->link = temp->link;
        printf("value %d is deleted", temp->data);
        free(temp);
}
void search() {
        struct node *temp = head;
        int pos = 0, found = 0, val;
        if (head == NULL) {
                printf("\n Empty List \n");
                return;
        }
```

10. When choice == 6
    - 2. Call deleteLocation()
    - Break
11. When choice == 7
    - Call search()
    - Break
12. When choice == 8
    - Call display()
    - Break
13. When choice == 9
    - Display exit
    - Exit(0)
    - Break
14. Default:
    - Display enter valid choice
15. Stop

OUTPUT:

SINGLY LINKED LIST
1-> InsertFirst
2-> InsertLast
3-> Insert Location
4-> Delete first
5-> Delete last
6-> Delete location
7-> Search
8-> Display
9-> Exit
Enter Choice:
1
Enter the value to insert to Front
21
Element inserted 21
Enter Choice:
2
Enter the element to insert Last
31
element inserted successfully 31
Enter Choice:
3
Enter the key were after you want to add Element
31
Enter the Element to inserted
41
value inserted successfully 41

```
        printf("\n Enter the value to search");
        scanf("%d", &val);
        while (temp != NULL) {
                if (temp->data == val) {
                        printf("%d value found at location %d \n", temp->data, pos+1);
                        found = 1;
                }
                pos++;
                temp = temp->link;
        }
        if (!found) {
                printf("Value %d not exist", val);
        }
}
void display() {
        struct node *temp = head;
        if (temp == NULL) {
                printf("\n List Empty");
                return;
        }
        printf("\n Elements in the List \n");
        while (temp != NULL) {
                printf("%d ", temp->data);
                temp = temp->link;
        }
}
void main() {
        int choice;
        printf("\n SINGLY LINKED LIST \n");
        do {
                printf("\n 1-> InsertFirst \n 2-> InsertLast \n 3-> Insert Location \n 4->
Delete first \n 5-> Delete last \n 6-> Delete location \n 7-> Search \n 8-> Display \n 9->
Exit");
                printf("\n Enter Choice: \n");
                scanf("%d", &choice);
                switch (choice) {
                        case 1: insertFirst();
                                break;
                        case 2: insertLast();
                                break;
                        case 3: insertLocation();
                                break;
                        case 4: deleteFirst();
                                break;
                        case 5: deleteLast();
                                break;
                        case 6: deleteLocation();
```

```
Enter Choice:
8
Elements in the List
21 31 41
Enter Choice:
7
Enter the value to search 41
41 value found at location 3
Enter Choice:
4
Value deleted 21
Enter Choice:
5
value 41 deleted
Enter Choice:
6
Enter the key that you want to delete
31
value 31 is deleted
Enter Choice:
9
Exit
```

```
                    break;
                case 7: search();
                    break;
                case 8: display();
                    break;
                case 9: printf("\n Exit \n");
                    exit(0);
                default: printf("\n INVALID CHOICE \n");
            }

        } while (choice != 9);
    }
```

RESULT:

Program is executed successfully and output is obtained.

ALGORITHM:

push()

1. Start
2. Create newnode and allocate memory for newnode using malloc
3. Read the element to be inserted and set newnode→link = NULL
4. Check whether top == NULL
    - If true go to step 5
    - Otherwise go to step 6
5. Set top = newnode
6. Set newnode→link = top and top = newnode
7. Display the inserted element
8. Stop

pop()

1. Start
2. Check whether top == NULL is true
    - If true go to step 3
    - Otherwise go to step 4
3. Display stack is empty and return
4. Set top = temp
5. Set top = temp→link
6. Display deleted element
7. Free temp
8. Stop

peek()

1. Start
2. Set temp = top
3. Check whether temp == NULL is true
    - If true display stack underflow and return
4. Display top element as temp→data
5. Stop

DATE: 15/10/2025

EXPERIMENT: 9.1

SINGLY LINKED STACK

AIM:

Write a C program to implement stack operations using singly linked list.

SOURCE CODE:

```
#include<stdio.h>
#include<stdlib.h>
struct node {
        int data;
        struct node *link;
};
struct node *top = NULL;

void push() {
        struct node *newnode;
        newnode = (struct node*)malloc(sizeof(struct node));
        if (newnode == NULL) {
                printf("\n No space avialble\n");
                return;
        }
        newnode->link = NULL;
        printf("\n Enter the element to insert: ");
        scanf("%d",&newnode->data);
        if (top == NULL) {
                top = newnode;
        }
        else {
                newnode->link = top;
                top = newnode;
        }
        printf("\n %d INSERTED SUCCESFULLY", newnode->data);
}
void pop() {
        struct node *temp = top;
        if (top == NULL) {
                printf("\n STACK UNDER FLOW");
                return;

        }
        printf("\n %d is popped ", temp->data);
        top = temp->link;
        free(temp);
}
```

display()

1. Start
2. Set temp = top
3. Check whether temp == NULL
    - If true display stack is empty and return
4. While temp != NULL is true go to step 5 otherwise go to step 7
5. Display temp→data
6. Set temp = temp→link
7. Stop

search()

1. Start
2. Set temp = top, pos = 0, found = 0
3. Check whether top == NULL is true
    - If true display stack is empty and return
4. Read the element to be searched in 'key'
5. While temp != NULL is true go to step 6 otherwise go to step 10
6. Check whether temp→data == key
    - If true go to step 7 otherwise step 8
7. Display the position of key and set found = 1
8. Increment pos by 1
9. Set temp = temp→link
10. If !found is true display element not found
11. Stop

main()

1. Start
2. Perform the given operations until choice != 6 using do-while loop
3. Read choice
4. Check the value of choice
5. When choice == 1
    - Call push()
    - Break
6. When choice == 2
    - Call pop()
    - Break
7. When choice == 3
    - Call peek()
    - Break

```
void peek() {
        struct node *temp = top;
        if (top == NULL) {
                printf("\n STACK UNDER FLOW");
                return;
        }
        printf("Top Element is %d", temp->data);
}

void display() {
        struct node *temp = top;
        if (top == NULL) {
                printf("\n NO ELEMENTS");
                return;
        }
        printf("\n ELEMENTS IN STACK ARE: \n");
        while (temp != NULL) {
                printf("%d ", temp->data);
                temp = temp->link;
        }
}

void search() {
        struct node *temp = top;
        int key, found = 0;
        if (top == NULL) {
                printf("\n STACK UNDERFLOW \n");
                return;
        }
        printf("\n ENTER THE ELEMENT TO SEARCH \n");
        scanf("%d",&key);
        while (temp != NULL) {
                if (temp->data == key) {
                        printf("\n %d ELEMENT FOUNDED \n", temp->data);
                        found = 1;
                }
                temp = temp->link;
        }
        if (!found) {
                printf("\n ELEMENT NOT FOUND");
        }
}
```

8. When choice == 4
    - Call display()
    - Break
9. When choice == 5
    - Call search()
    - Break
10. When choice == 6
11. Display exit
12. Exit(0)
13. Break
14. Default: Display enter valid choice
15. Stop

OUTPUT:

```
*****STACK*****
1->Push()
2->Pop()
3->Peek()
4->Display()
5->Search()
6->EXIT
ENTER THE CHOICE: 1
Enter the element to insert: 21
21 INSERTED SUCCESFULLY
ENTER THE CHOICE: 1
Enter the element to insert: 25
25 INSERTED SUCCESFULLY
ENTER THE CHOICE: 1
Enter the element to insert: 56
56 INSERTED SUCCESFULLY
ENTER THE CHOICE: 3
Top Element is 56
ENTER THE CHOICE: 4
ELEMENTS IN STACK ARE:
56 25 21
ENTER THE CHOICE: 2
56 is popped
ENTER THE CHOICE: 5
ENTER THE ELEMENT TO SEARCH: 25
25 ELEMENT FOUNDED
ENTER THE CHOICE: 6
EXIT
```

```
void main() {
        int choice;

        do {
                printf("\n *****STACK*****\n");
                printf("\n 1->Push() \n 2->Pop() \n 3->Peek() \n 4->Display() \n 5->Search()
\n 6->EXIT");
                printf("\n ENTER THE CHOICE: ");
                scanf("%d",&choice);

                switch(choice) {
                        case 1: push();
                                break;
                        case 2: pop();
                                break;
                        case 3: peek();
                                break;
                        case 4: display();
                                break;
                        case 5: search();
                                break;
                        case 6: printf("\n EXIT \n");
                                break;
                        default: printf("Enter a valid Choice");
                }

        } while(choice != 6);
}
```

RESULT:

Program is executed successfully and output is obtained.

ALGORITHM:

enqueue()

1. Start
2. Create newnode and allocate memory for newnode using malloc
3. Set newnode→link = NULL and temp = head
4. Check whether head == NULL
    - If true go to step 5
    - Otherwise go to step 6
5. Set head = newnode
6. While temp→link != NULL is true go to step 7 otherwise go to step 8
7. Set temp = temp→link
8. Read the element to be inserted
9. Set temp→link = newnode
10. Display newly inserted element
11. Stop

dequeue()

1. Start
2. Check whether head == NULL is true
    - If true go to step 3 otherwise go to step 4
3. Display queue is empty and return
4. Set head = temp
5. Set head = temp→link
6. Display deleted element
7. Free temp
8. Stop

display()

1. Start
2. Set temp = head
3. Check whether temp == NULL is true
    - Display queue is empty and return
4. While temp != NULL is true go to step 5 otherwise go to step 7
5. Display temp→data
6. Set temp = temp→link
7. Stop

DATE: 15/10/2025

EXPERIMENT: 9.2

SINGLY LINKED QUEUE

AIM:

Write a C program to implement queue operations using singly linked list.

SOURCE CODE:

```
#include<stdio.h>
#include<stdlib.h>
struct node {
        int data;
        struct node* link;
};
struct node *head = NULL;
void enqueue() {
        struct node *temp = head, *newnode;
        newnode = (struct node*)malloc(sizeof(struct node));
        if (newnode == NULL) {
                printf("\n NO SPACE AVAILABLE \n");
                return;
         }
        newnode->link = NULL;
        printf("\n ENTER THE ELEMENT TO INSERT: ");
        scanf("%d", &newnode->data);
        if (head == NULL) {
                head = newnode;
        }
        else {
                while (temp->link != NULL) {
                        temp = temp->link;
                }
        temp->link = newnode;
        }
        printf("\n %d ELEMENT INSERTED SUCCESSFULLY \n", newnode->data);
}
void dequeue() {
        struct node *temp = head;
        if (head == NULL) {
                printf("\n NO ELEMENTS \n");
                return;
        }
        printf("\n %d IS DELETED \n", temp->data);
        head = temp->link;
   free(temp);
}
```

search()

1. Start
2. Set temp = head, pos = 0 and found = 0
3. Check whether head == NULL is true
    - Display queue is empty and return
4. Read the element to be searched in 'val'
5. While temp != NULL is true go to step 6 otherwise go to step 10
6. Check whether temp→data == val is true
    - Go to step 7 otherwise go to step 8
7. Display the position of val and set found = 1
8. Increment pos by 1
9. Set temp = temp→link
10. If !found is true display value does not exist
11. Stop

main()

1. Start
2. Perform the given operations until choice != 5 using do-while loop
3. Read choice
4. Read the value of choice
5. When choice == 1 → Call enqueue()
6. When choice == 2 → Call dequeue()
7. When choice == 3 → Call display()
8. When choice == 4 → Call search()
9. When choice == 5 → Display exit and terminate
10. Default → Display enter valid choice
11. Stop

OUTPUT:

```
*****QUEUE*****
1->Enqueue()
2->Dequeue()
3->Display()
4->Search()
5->EXIT
ENTER THE CHOICE: 1
ENTER THE ELEMENT TO INSERT: 23
23 ELEMENT INSERTED SUCCESSFULLY
```

```
void display() {
	struct node *temp = head;
	if (head == NULL) {
		printf("\n NO ELEMENTS");
		return;
	}
	printf("\n ELEMENTS IN QUEUE ARE \n");
	while (temp != NULL) {
		printf("%d ", temp->data);
		temp = temp->link;
	}
}

void search() {
	struct node *temp = head;
	int key, pos = 0, found = 0;
	if (head == NULL) {
		printf("\n QUEUE EMPTY\n");
		return;
	}
	printf("\n ENTER THE ELEMENT TO SEARCH \n");
	scanf("%d", &key);
	while (temp != NULL) {
		if (temp->data == key) {
			printf("\n %d ELEMENT FOUND AT %d \n", temp->data, pos);
			found = 1;
		}
		temp = temp->link;
		pos++;
	}
	if (!found) {
		printf("\n ELEMENT NOT FOUND");
	}
}

int main() {
	int choice;
	do {
		printf("\n *****QUEUE*****\n");
		printf("\n 1->Enqueue() \n 2->Dequeue() \n 3->Display() \n 4->Search() \n 5-
>EXIT");
		printf("\n ENTER THE CHOICE \n");
		scanf("%d", &choice);
		switch (choice) {
			case 1: enqueue();
				break;
```

ENTER THE CHOICE:1
ENTER THE ELEMENT TO INSERT: 34
34 ELEMENT INSERTED SUCCESSFULLY
ENTER THE ELEMENT TO INSERT:56
56 ELEMENT INSERTED SUCCESSFULLY
ENTER THE CHOICE: 3
ELEMENTS IN QUEUE ARE
23 34 56
ENTER THE CHOICE: 4
ENTER THE ELEMENT TO SEARCH: 34
34 ELEMENT FOUND AT POSITION 2
ENTER THE CHOICE: 2
23 IS DELETED
ENTER THE CHOICE:5
EXIT

```
                case 2: dequeue();
                        break;
                case 3: display();
                        break;
                case 4: search();
                        break;
                case 5: printf("\n EXIT \n");
                        break;
                default: printf("Enter a valid Choice");
            }
        } while (choice != 5);
        return 0;
}
```

RESULT:

Program is executed successfully and output is obtained.

ALGORITHM:

insertFirst()

1. Start
2. Create a new node and allocate memory for it.
3. Check if the list is empty (head == NULL), if true set head to the new node.
4. Otherwise:
    - Set the Rlink of the new node to point to the current head.
    - Set the Llink of the current head to point to the new node.
    - Update head to point to the new node.
5. Update head to point to the new node.
6. Print a success message indicating that the element was inserted successfully.
7. Stop

insertLast()

1. Start
2. Create a new node and allocate memory for it.
3. Check if the list is empty (head == NULL), if true set head to the new node.
4. Otherwise:
    - Traverse the list until reaching the last node (where Rlink == NULL).
    - Set the Llink of the new node to the last node and the Rlink of the last node to the new node.
5. Set the Llink of the new node to the last node and the Rlink of the last node to the new node.
6. Print a success message indicating that the element was inserted successfully.
7. Stop

insertLocation()

1. Start
2. Create a new node and allocate memory for it. Set newnode->Rlink, Llink = NULL
3. Check if the list is empty (head == NULL).
    - If the list is empty, print "List is empty" and exit.
4. Start from the head and traverse through the list node by node until the key is found.
5. If the key is not found:
    - Print an error message: "Key not found" and exit.
6. If the key is in the first node:
    - Set the new node's Rlink to the current head.
    - Set the Llink of the current head to point to the new node.

DATE: 22/10/2025

EXPERIMENT: 10

DOUBLY LINKED LIST

AIM:

Write a C program to implement queue operations using singly linked list.

SOURCE CODE:

```
#include<stdio.h>
#include<stdlib.h>
#include<malloc.h>
struct node {
        int data;
        struct node *Llink;
        struct node *Rlink;
};
struct node *head = NULL;
void insertFirst() {
        struct node *newnode;
        newnode = (struct node*)malloc(sizeof(struct node));
        if (newnode == NULL) {
                printf("\n No Space available");
                return;
        }
        newnode->Llink = NULL;
        newnode->Rlink = NULL;
        printf("\n Enter the element to insert: ");
        scanf("%d",&newnode->data);
        if (head == NULL) {
                head = newnode;
        }
        else {
                newnode->Rlink = head;
                head->Llink = newnode;
                head = newnode;
        }
        printf("\n %d inserted sucessfully", newnode->data);
}
void insertLast() {
        struct node *newnode, *temp = head;
        newnode = (struct node*)malloc(sizeof(struct node));
        if (newnode == NULL) {
                printf("\n Error: No space available for a new node.\n");
                return;
        }
```

- Update head to point to the new node.

7. If the key is in the middle node:
    - Set the new node's Llink to the current node.
    - Set the new node's Rlink to the next node.
    - Update the current node's Rlink to point to the new node.
8. Update the next node's Llink to point to the new node.
9. If the key is in the last node:
    - Set the new node's Llink to the current node.
    - Set the new node's Rlink to NULL.
    - Update the current node's Rlink to point to the new node.
10. Print a success message: "Node inserted successfully."
11. Stop

deleteFirst()

1. Start
2. Check if the list is empty (head == NULL):
    - If empty, print "LIST EMPTY" and return.
3. Set a temporary pointer temp to the head node.
4. Print the message showing the data of the deleted node (temp->data).
5. If the Rlink of the head node is NULL (single node), set head = NULL.
6. Otherwise:
    - Set the next node (nxt = temp->Rlink).
    - Update head = nxt.
    - Set nxt->Llink = NULL.
7. Free the memory occupied by the deleted node (free(temp)).
8. Stop

deleteLast()

1. Start
2. Check if the list is empty (head == NULL).
    - If empty, print "LIST IS EMPTY" and return.
3. If the list has only one node:
    - Print the message showing the data of the deleted node (temp->data).
    - Set head = NULL.
4. If the list has more than one node:
    - Traverse the list until the last node (temp->Rlink == NULL).
    - Print the message showing the data of the deleted node (temp->data).
    - Set the previous node (nxt = temp->Llink).

```
    newnode->Llink = NULL;
    newnode->Rlink = NULL;
    printf("\n Enter the element to insert: ");
    scanf("%d",&newnode->data);
    if (head == NULL) {
        head = newnode;
    }
   else {
        while (temp->Rlink != NULL) {
                temp = temp->Rlink;
        }
        temp->Rlink = newnode;
        newnode->Llink = temp;
   }
   printf("%d inserted succesfully", newnode->data);
}
void insertLocation() {
        struct node *newnode, *temp = head, *nxt;
        int key;
        newnode = (struct node*)malloc(sizeof(struct node));
        if (newnode == NULL) {
                printf("\n Error: No space available for a new node.\n");
                return;
        }
        if (head == NULL) {
                printf("\n List is empty \n");
        }
        else {
                printf("\n Enter the key were after you want to insert the element: ");
                scanf("%d",&key);
                while (temp != NULL && temp->data != key) {
                        temp = temp->Rlink;
                }
                if (temp == NULL) {
                printf("\n NO ELEMENT FOUND \n");
                return;
        }
        printf("\n Enter the element to insert: ");
        scanf("%d",&newnode->data);
        if (temp->Rlink == NULL) {
                newnode->Llink = temp;
                newnode->Rlink = NULL;
                temp->Rlink = newnode;
        }
        else {
                nxt = temp->Rlink;
                newnode->Llink = temp;
                newnode->Rlink = nxt;
                temp->Rlink = newnode;
                nxt->Llink = newnode;
        }
        printf("%d inserted succesfully", newnode->data);
        }
}
```

- Update the Rlink of the previous node to NULL (nxt->Rlink = NULL).

5. Free the memory occupied by the deleted node (free(temp)).
6. Stop

deleteLocation()

1. Start
2. Check if the list is empty (head == NULL).
    - If empty, print "LIST IS EMPTY" and return.
3. Prompt the user to enter the key of the node to delete.
4. Traverse the list to find the node containing the specified key.
5. If the key is not found:
    - Print "NO ELEMENT FOUND" and return.
6. If the node is the only node:
    - Set head = temp->Rlink.
    - If head != NULL, set head->Llink = NULL.
7. If the node is the last node:
    - Set temp->Llink->Rlink = NULL.
8. If the node is a middle node:
    - Set prev = temp->Llink and next = temp->Rlink.
    - Update prev->Rlink = next and next->Llink = prev.
9. Print the message: "{key} Deleted successfully".
10. Free the memory occupied by the node: free(temp).
11. Stop

display()

1. Start
2. Check if the list is empty (head == NULL):
    - If the list is empty, print the message: "No elements in list" and return.
3. Otherwise:
4. Traverse the list starting from the head node:
    - Print the message: "Elements in List".
    - Initialize a temporary pointer (temp) to the head node.
    - Traverse through the list using a while loop:
    - Print the data of the current node (temp->data).
    - Move the temporary pointer to the next node (temp = temp->Rlink).
5. Continue this process until the entire list has been displayed (when temp == NULL).
6. Stop

```
void deleteFirst() {
        struct node *temp = head, *nxt;
        if (head == NULL) {
                printf("\n LIST EMPTY \n");
                return;
        }
        printf("\n %d is deleted", temp->data);
        if (temp->Rlink == NULL) {
                head = NULL;
        }
        else {
                nxt = temp->Rlink;
                head = nxt;
                nxt->Llink = NULL;
        }
        free(temp);
}

void deleteLast() {
        struct node *temp = head, *nxt;
        if (head == NULL) {
                printf("\n LIST IS EMPTY \n");
                return;
        }
        if (temp->Rlink == NULL) {
                printf("\n %d is deleted", temp->data);
                head = NULL;
        }
        else {
                while (temp->Rlink != NULL) {
                        temp = temp->Rlink;
                }
                printf("\n %d is deleted", temp->data);
                nxt = temp->Llink;
                nxt->Rlink = NULL;
        }
        free(temp);
}
```

search()

1. Start
2. Check if the list is empty (head == NULL).
    - If empty, print "LIST EMPTY" and return.
3. Read the key from the user to search.
4. Traverse the list, starting from the head:
    - For each node, compare its data with the given key.
    - If the key is found, print the message: "{key} is found at location {position}" and set the found flag to 1.
    - Keep track of the position as you traverse the list.
5. If the found flag is still 0, print: "ELEMENT NOT FOUND".
6. Stop

main()

1. Start
2. Declare a variable choice to store the user's input.
3. Display Menu
4. Input User Choice
5. Execute Based on Choice
    - Case 1: Call insertFirst()
    - Case 2: Call insertLast()
    - Case 3: Call insertLocation()
    - Case 4: Call deleteFirst()
    - Case 5: Call deleteLast()
    - Case 6: Call deleteLocation()
    - Case 7: Call search()
    - Case 8: Call display()
    - Case 9: Print "EXIT" and terminate the program
6. Repeat Until Exit
7. Stop

```
void deleteLocation() {
        struct node *temp = head, *prev, *next;
        int key;
        if (head == NULL) {
                printf("\n LIST IS EMPTY \n");
                return;
         }
        printf("\n Enter the key which you want to delete: \n");
        scanf("%d", &key);
        while (temp != NULL && temp->data != key) {
                temp = temp->Rlink;
        }
        if (temp == NULL) {
                printf("\n NO ELEMENT FOUND \n");
                return;
        }
        if (temp->Llink == NULL) {
                head = temp->Rlink;
                if (head != NULL) {
                        head->Llink = NULL;
                        }
        }
        else if (temp->Rlink == NULL) {
                temp->Llink->Rlink = NULL;
        }
        else {
                prev = temp->Llink;
                next = temp->Rlink;
                prev->Rlink = next;
                next->Llink = prev;
        }
        printf("%d Deleted successfully\n", temp->data);
        free(temp);
}
void display() {
        struct node *temp = head;
        if (head == NULL) {
                printf("\n NO ELEMENTS IN LIST ");
                return;
        }
        printf("\n ** ELEMENTS IN LIST ** \n");
        while (temp != NULL) {
                printf("%d ", temp->data);
                temp = temp->Rlink;
        }
}
```

OUTPUT:

*****DOUBLY LINKEDLIST ****
1-> insert First
2->insert Last
3->insert Location
4->delete First
5->delete Last
6->delete Location
7-> Search
8->Display
9->Exit
Enter the choice: 1
Enter the element to insert: 12
12 inserted successfully
Enter the choice 2
Enter the element to insert24
24 inserted successfully
Enter the choice: 3
Enter the key were after you want to insert the element: 12
Enter the element to insert: 18
18 inserted successfully
Enter the choice:8
** ELEMENTS IN LIST **
12 18 24
Enter the choice: 7
Enter the key to search: 12
12 is found at location 0
Enter the choice: 4
12 is deleted
Enter the choice: 5
24 is deleted
Enter the choice 6
Enter the key which you want to delete: 18
18 Deleted successfully
Enter the choice: 9
EXIT

```
void search() {
        int key, pos = 0, found = 0;
        struct node *temp = head;
        if (head == NULL) {
                printf("\n LIST EMPTY \n");
                return;
        }
        printf("\n Enter the key to search");
        scanf("%d",&key);
        while (temp != NULL) {
                if (temp->data == key) {
                        printf("%d is found at location %d", temp->data, pos);
                        found = 1;
                }
                temp = temp->Rlink;
                pos++;
        }
        if (!found) {
                printf("\n ELEMENT NOT FOUND \n");
        }
}
void main() {
        int choice;
        do {
                printf("\n*****DOUBLY LINKEDLIST ****\n");
                printf("\n 1-> insert First \n 2->insert Last \n 3->insert Location \n 4->delete
First \n 5->delete Last \n 6->delete Location \n 7-> Search \n 8->Display \n 9->Exit \n");
                printf("Enter the choice ");
                scanf("%d",&choice);
                switch(choice) {
                        case 1: insertFirst();
                                break;
                        case 2: insertLast();
                                break;
                        case 3: insertLocation();
                                break;
                        case 4: deleteFirst();
                                break;
                        case 5: deleteLast();
                                break;
                        case 6: deleteLocation();
                                break;
                        case 7: search();
                                break;
                        case 8: display();
                                break;
```

```
                        case 9: printf("\nEXIT\n");
                                exit(0);
                        default: printf("enter valid choice");
                                break;
                }

        } while(choice != 9);
}
```

RESULT:

Program is executed successfully and output is obtained.

ALGORITHM:

1. Start.
2. Declare sets U[5], A[5], B[5], uni[5], ints[5], diffA[5], diffB[5], compA[5], compB[5].
3. Check if i < 5, if true goto step 4 otherwise goto step 5.
4. If A[i] == 1, display U[i].
5. Check if i < 5, if true goto step 6 otherwise goto step 7.
6. If B[i] == 1, true display U[i].
7. Check if i < 5, if true goto step 8 otherwise goto step 10.
8. Perform uni[i] = A[i] | B[i].
9. Display uni[i].
10. Check if i < 5, if true goto step 11, otherwise goto step 12.
11. If uni[i] == 1 true, display U[i].
12. Check if i < 5, if true goto step 13, otherwise goto step 15.
13. Perform ints[i] = A[i] & B[i].
14. Display ints[i].
15. Check if i < 5, if true goto step 16 otherwise goto step 17.
16. If ints[i] == 1, is true display U[i].
17. Check whether i < 5, if true goto step 18 otherwise goto step 20.
18. Perform compA[i] = 1 − A[i].
19. Display compA[i].
20. Check whether i < 5, if true goto step 21 otherwise goto step 22.
21. If compA[i] == 1, true display U[i].
22. Check whether i < 5, if true goto step 23 otherwise goto step 25.
23. Perform compB[i] = 1 − B[i].
24. Display compB[i].
25. Check whether i < 5, if true goto step 26 otherwise goto step 27.
26. If compB[i] == 1, true display U[i].
27. Check whether i < 5, if true goto step 28 otherwise goto step 30.
28. Perform diffA[i] = A[i] & compB[i].
29. Display diffA[i].
30. Check whether i < 5, if true goto step 31 otherwise goto step 32.
31. Check whether diffA[i] == 1, display U[i].
32. Check whether i < 5, if true goto step 33 otherwise goto step 35.
33. Perform diffB[i] = B[i] & compA[i].
34. Display diffB[i].
35. Check whether i < 5, if true goto step 36 otherwise goto step 37.
36. Check whether diffB[i] == 1, true, display U[i].
37. Stop

DATE: 29/10/2025

EXPERIMENT: 11

IMPLEMENTATION OF SET

AIM:

Given a U={1,2,3,4,5}, A={1,4,5} and B={2,3,4}. Write a C program to find A union B, A intersection B, A-B, B-A in bit vector.

SOURCE CODE:

```
#include<stdio.h>
void main()
{
        int i;
        int U[5]={1,2,3,4,5};
        int A[5]={1,0,0,1,1};
        int B[5]={0,1,1,1,0};
        int uni[5],ints[5],diffB[5],diffA[5],compA[5],compB[5];
        // Display Universal Set
        printf("\n UNIVERSAL SET IS {");
        for(i=0;i<5;i++){
                printf("%d ",U[i]);
        }
        printf("} \n");
        //Display Set A
        printf("\n SET A {");
        for(i=0;i<5;i++){
                if(A[i]==1){
                        printf("%d ",U[i]);
                }
        }
        printf("} \n");
        //Display Set B
        printf("\n SET B {");
        for(i=0;i<5;i++){
                if(B[i]==1){
                        printf("%d ",U[i]);
                }
        }
        printf("} \n");
        //Union of A and B
        printf("Union of A and B in bit representation is=");
        for(i=0;i<5;i++){
                uni[i]=A[i] | B[i];
                printf("%d",uni[i]);
        }
        printf("\n UNION {");
```

OUTPUT:

UNIVERSAL SET IS {1 2 3 4 5 }

SET A {1 4 5 }

SET B {2 3 4 }

Union of A and B in bit representation is=11111

UNION {1 2 3 4 5 }

Intersection of A and B in bit representation is:00010

INTERSECTION {4 }

Complement of A is bit representation is=01100

A COMPLIMENT {2 3 }

Complement of B is bit representation is=10001

B COMPLIMENT {1 5 }

Difference of A-B in bit representation is=10001

A-B {1 5 }

Difference of B-A in bit representation is=01100

B-A {2 3 }

```
for(i=0;i<5;i++){
        if(uni[i]==1){
                printf("%d ",U[i]);
        }
}
printf("} \n");
//Intersection of A and B
printf("Intersection of A and B in bit representation is:");
for(i=0;i<5;i++){
        ints[i]=A[i] & B[i];
        printf("%d",ints[i]);
}
printf("\n INTERSECTION {");
for(i=0;i<5;i++){
        if(ints[i]==1) {
        printf("%d ",U[i]);
        }
}
printf("} \n");
//Complement of A
printf("Complement of A is bit representation is=");
for(i=0;i<5;i++){
        compA[i]=1-A[i];
        printf("%d",compA[i]);
}
printf("\n A COMPLIMENT {");
for(i=0;i<5;i++){
        if(compA[i]==1){
                printf("%d ",U[i]);
        }
}
printf("} \n");
//Complement of B
printf("Complement of B is bit representation is=");
for(i=0;i<5;i++){
        compB[i]=1-B[i];
        printf("%d",compB[i]);
}
printf("\n B COMPLIMENT {");

for(i=0;i<5;i++){
        if(compB[i]==1){
                printf("%d ",U[i]);
        }
}
```

```
        printf("} \n");
        //Difference of A-B
        printf("Difference of A-B in bit representation is=");
        for(i=0;i<5;i++){
                diffA[i]=A[i] & compB[i];
                printf("%d",diffA[i]);
        }
        printf("\n A-B {");
        for(i=0;i<5;i++){
            if(diffA[i]==1){
                    printf("%d ",U[i]);
            }
        }
        printf("} \n");
        //Difference of B-A
        printf("Difference of B-A in bit representation is=");
        for(i=0;i<5;i++){
            diffB[i]=B[i] & compA[i];
            printf("%d",diffB[i]);
        }
        printf("\n B-A {");
        for(i=0;i<5;i++){
            if(diffB[i]==1){
                    printf("%d ",U[i]);
            }
        }
        printf("} \n");
}
```

RESULT:

Program is executed successfully and output is obtained.

ALGORITHM:

Disjoint set

1. Start
2. Define MAX = 1000
3. Declare parent[MAX] array
4. Stop

initialize(int n)

1. Start
2. Check whether i < n if true, go to step 3 otherwise go to step 4
3. Set parent[i] = i
4. Stop

int find(int x)

1. Start
2. While x != parent[x] is true, go to step 3 otherwise go to step 4
3. Set x = parent[x]
4. Return x
5. Stop

union_sets(int x, int y)

1. Start
2. Set int root x = find(x) and int root y = find(y)
3. If root x != root y true, go to step 4 otherwise go to step 5
4. Set parent[root x] = root y
5. Stop

connected(int x, int y)

1. Start
2. Declare n = 10
3. Call the function initialize(n)
4. Call the function union_sets(1,2), union_sets(3,4) and union_sets(2,3)
5. Display connected(1,4) and connected(1,5)
6. Stop

OUTPUT:

Are 1 and 4 connected? Yes

Are 1 and 5 connected? No

DATE: 29/10/2025

EXPERIMENT: 12

DISJOINT SET OPERATIONS

AIM:

Write a program to implement disjoint set and its operations.

SOURCE CODE:

```
#include <stdio.h>
#define MAX 1000
int Parent[MAX];
void initialize(int n) {
        for(int i=0;i<n;i++)
        {
                Parent[i]=i;
        }
}
int find(int x) {
        while(x!=Parent[x]) {
                x=Parent[x];
        }
        return x;
}
void union_set(int x,int y) {
        int rootX=find(x);
        int rootY=find(y);
        if(rootX!=rootY) {
                Parent[rootX]=rootY;
        }
}
int Connected(int x,int y) {
        return find(x)==find(y);
}
int main() {
        int n=10;
        initialize(n);
        union_set(1,2);
        union_set(3,4);
        union_set(2,3);
        printf("Are 1 and 4 connected ?%s \n",Connected(1,4)? "Yes":"No");
        printf("Are 1 and 5 connected ?%s \n",Connected(1,5)? "Yes":"No");
        return 0;
}
```

RESULT:

Program is executed successfully and output is obtained.

ALGORITHM:

enqueue()

1. Start
2. Check whether rear == 99
    - If true, display "Queue Overflow" and return
3. Otherwise:
4. Check whether front == -1
    - If true, set front = 0
5. Increment rear by 1
6. Set queue[rear] = value
7. Stop

dequeue()

1. Start
2. Check whether front == -1 OR front > rear
    - If true, return -1
3. Otherwise:
4. Return queue[front]
5. Increment front = front + 1
6. Stop

bfs(adj, start, n)

1. Start
2. Call enqueue(start)
3. Set visited[start] = 1
4. Display "BFS Traversal:"
5. While front <= rear is true, repeat steps 6–12
6. Call dequeue() and store the returned value in current
7. Display current
8. Set i = 0
9. Check whether i < n
    - If true, go to step 10
    - Otherwise, go to step 12
10. Check whether
    - adj[current][i] == 1 and
    - visited[i] == 0

DATE: 05/11/2025

EXPERIMENT: 13

BREADTH-FIRST SEARCH (BFS)

AIM:

Write a program to implement Breadth-First Search (BFS) for graph traversal using an adjacency matrix.

SOURCE CODE:

```
#include<stdio.h>
#include<stdlib.h>
int queue[100], front = -1, rear = -1;
int visited[100];

void enqueue(int value) {
        if (rear == 99)
                printf("Queue Overflow\n");
        else {
                if (front == -1)
                        front = 0;
                queue[++rear] = value;
        }
}

int dequeue() {
        if (front == -1 || front > rear)
                return -1;
        return queue[front++];
}

void bfs(int adj[10][10], int start, int n) {
        int i;
        enqueue(start);
        visited[start] = 1;
        printf("BFS Traversal: ");
        while (front <= rear) {
                int current = dequeue();
                printf("%d ", current);
                for (i = 0; i < n; i++) {
                        if (adj[current][i] == 1 && !visited[i]) {
                                enqueue(i);
                                visited[i] = 1;
                        }
                }
        }
}
```

If true:

- Call enqueue(i)
- Set visited[i] = 1

11. Increment i = i + 1 and go back to step 9
12. Continue loop until queue becomes empty
13. Stop

main()

1. Start
2. Declare variables n, adj[][], i, j, start
3. Display "Enter number of vertices:"
4. Read value of n
5. Display "Enter adjacency matrix:"
6. For i = 0 to n-1, repeat steps 7–8
7. For j = 0 to n-1, read adj[i][j]
8. End loops
9. Display "Enter starting vertex:"
10. Read value of start
11. Call bfs(adj, start, n)
12. Stop

OUTPUT:

```
Enter number of vertices: 5
Enter adjacency matrix:
0 1 1 0 0
1 0 1 1 0
1 1 0 0 1
0 1 0 0 1
0 0 1 1 0
Enter starting vertex (0 to 4): 0
BFS Traversal: 0 1 2 3 4
```

```
int main() {
        int n, adj[10][10], i, j, start;
        printf("Enter number of vertices: ");
        scanf("%d", &n);
        printf("Enter adjacency matrix:\n");
        for (i=0; i<n; i++)
                for (j=0; j<n; j++)
                        scanf("%d", &adj[i][j]);
        printf("Enter starting vertex (0 to %d): ", n-1);
        scanf("%d", &start);
        bfs(adj, start, n);
        return 0;
}
```

RESULT:

Program is executed successfully and output is obtained.

ALGORITHM:

push()

1. Start
2. Increment top = top + 1
3. Assign stack[top] = value
4. Stop

pop()

1. Start
2. Check whether top == -1
    - If true → return -1
3. Otherwise:
4. Return stack[top]
5. Decrement top = top – 1
6. Stop

dfs_non_recursive(adj, start, n)

1. Start
2. Call push(start)
3. Set visited[start] = 1
4. Display "DFS Traversal (non-recursive):"
5. Repeat steps 6–12 while top != -1
6. Call pop() and store the returned value in current
7. Display current
8. Set i = n – 1
9. Check whether i >= 0
    - If true → go to step 10
    - Else → go to step 12
10. Check if
    - adj[current][i] == 1 AND
    - visited[i] == 0
      If both conditions true:
    - Call push(i)
    - Set visited[i] = 1
11. Decrement i = i – 1 and return to step 9
12. Continue loop until stack becomes empty
13. Stop

DATE: 05/11/2025

EXPERIMENT: 14

DEPTH-FIRST SEARCH (DFS)

AIM:

Write a program to implement Depth-First Search (DFS) for graph traversal using an adjacency matrix.

SOURCE CODE:

```
#include<stdio.h>
int stack[100], top = -1;
int visited[100];

void push(int value) {
        stack[++top] = value;
}

int pop() {
        if (top == -1)
                return -1;
        else
                return stack[top--];
}

void dfs_non_recursive(int adj[10][10], int start, int n) {
        int i;
        push(start);
        visited[start] = 1;
        printf("DFS Traversal (non-recursive): ");
        while (top != -1) {
                int current = pop();
                printf("%d ", current);
                for (i = n-1; i >= 0; i--) {
                        if (adj[current][i] == 1 && !visited[i]) {
                                push(i);
                                visited[i] = 1;
                        }
                }
        }
}
```

main()

1. Start
2. Declare variables n, adj[][], i, j, start
3. Display "Enter number of vertices:"
4. Read n
5. Display "Enter adjacency matrix:"
6. For i = 0 to n – 1:
    - For j = 0 to n – 1:
        - Read adj[i][j]
7. Display "Enter starting vertex:"
8. Read start
9. Call dfs_non_recursive(adj, start, n)
10. Stop

OUTPUT:

Enter number of vertices: 5

Enter adjacency matrix:

0 1 1 0 0

1 0 1 1 0

1 1 0 0 1

0 1 0 0 1

0 0 1 1 0

Enter starting vertex (0 to 4): 0

DFS Traversal (non-recursive): 0 1 3 4 2

```
int main() {
        int n, adj[10][10], i, j, start;
        printf("Enter number of vertices: ");
        scanf("%d", &n);

        printf("Enter adjacency matrix:\n");
        for (i=0; i<n; i++)
                for (j=0; j<n; j++)
                        scanf("%d", &adj[i][j]);

        printf("Enter starting vertex (0 to %d): ", n-1);
        scanf("%d", &start);

        dfs_non_recursive(adj, start, n);

        return 0;
}
```

RESULT:

Program is executed successfully and output is obtained.

ALGORITHM:

addEdge(int u, int v)

1. Start
2. Set adj[u][v] = 1
3. Stop

dfs(int v, int n)

1. Start
2. Set visited[v] = 1
3. Set i = 0
4. Check whether i < n
    - If true, go to step 5
    - Else go to step 7
5. Check if adj[v][i] && !visited[i] is true
    - If true, call dfs(i, n)
6. Increment i by 1 and go to step 4
7. Set stack[++top] = v
8. Stop

topologicalSort(int n)

1. Start
2. Set i = 0
3. Check whether i < n
    - If true, go to step 4
    - Else go to step 6
4. Check if !visited[i] is true
    - If true, call dfs(i, n)
5. Increment i by 1 and go to step 3
6. Display the topological order
7. While top >= 0 is true
    - Display stack[top--]
8. Stop

main()

1. Start
2. Declare the number of vertices to variable n
3. Read the number of vertices into variable n
4. Check whether i < n (loop init) → go to step 5
5. Check whether j < n → go to step 6, else go to step 7
6. Set adj[i][j] = 0

DATE: 12/11/2025

EXPERIMENT: 15

TOPOLOGICAL SORT

AIM:

Write a C program for performing topological sort.

SOURCE CODE:

```
#include <stdio.h>
#include <stdlib.h>
#define MAX 100
int adj[MAX][MAX];
int visited[MAX];
int stack[MAX];
int top = -1;
void addEdge(int u, int v) {
                    adj[u][v] = 1;
}
void dfs(int v, int n) {
   visited[v] = 1;
   for (int i = 0; i < n; i++) {
      if (adj[v][i] && !visited[i]) {
         dfs(i, n);
      }
   }
   stack[++top] = v;
}

void topologicalSort(int n) {
   for (int i = 0; i < n; i++) {
      if (!visited[i]) {
         dfs(i, n);
      }
   }
   printf("Topological Order: ");
   while (top >= 0) {
       printf("%d ", stack[top--]);
   }
   printf("\n");
 }
```

main()

1. Start
2. Declare the number of vertices to variable n
3. Read the number of vertices into variable n
4. Check whether i < n (loop init) → go to step 5
5. Check whether j < n → go to step 6, else go to step 7
6. Set adj[i][j] = 0
7. Set visited[i] = 0
8. Read the number of edges into variable edges
9. Read each edge using a loop
10. Check whether i < edges → go to step 11 else go to step 13
11. Read u and v
12. Call addEdge(u, v)
13. Call topologicalSort(n)
14. Stop

OUTPUT:

Enter the number of vertices: 6
Enter the number of edges: 6
Enter the edges (u v) (0-based index):
5 2
5 0
4 0
4 1
2 3
3 1
Topological Order: 5 4 2 3 1 0

```
int main()
{
    int n, edges, u, v;
    printf("Enter the number of vertices: ");
    scanf("%d", &n);
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            adj[i][j] = 0;
        }
        visited[i] = 0;
    }
    printf("Enter the number of edges: ");
    scanf("%d", &edges);
    printf("Enter the edges (u v) (0-based index):\n");
    for (int i = 0; i < edges; i++) {
        scanf("%d %d", &u, &v);
        addEdge(u, v);
    }
    topologicalSort(n);
    return 0;
}
```

RESULT:

Program is executed successfully and output is obtained.

ALGORITHM:

main()

1. Start
2. Display “Implementation of Kruskal's algorithm”
3. Read the number of vertices n
4. For every i from 1 to n
    - For every j from 1 to n
        - Read cost[i][j]
        - If cost[i][j] == 0, set cost[i][j] = 999
5. Display “The edges of Minimum Cost Spanning Tree are”
6. Repeat steps 7–14 while ne < n
7. Set min = 999
8. For every i = 1 to n
    - For every j = 1 to n
        - If cost[i][j] < min
            - Set min = cost[i][j]
            - Set a = u = i
            - Set b = v = j
9. Set u = find(u)
10. Set v = find(v)
11. Check whether uni(u, v) returns 1
    - If true:
        - Print neth edge as (a, b) with weight min
        - Increase ne by 1
        - Add min to mincost
12. Set cost[a][b] = 999
13. Set cost[b][a] = 999
14. Continue loop until ne == n
15. Display “Minimum cost = mincost”
16. Stop

DATE: 19/11/2025

EXPERIMENT: 16

KRUSKAL'S ALGORITHM

AIM:

Write a program to implementing Kruskal's algorithm.

SOURCE CODE:

```
#include<stdio.h>
#include<stdlib.h>
int i, j, k, a, b, u, v, n, ne = 1;
int min, mincost = 0, cost[9][9], parent[9];
int find(int);
int uni(int, int);

void main() {
 printf("\nImplementation of Kruskal's algorithm\n");
 printf("\nEnter the no. of vertices:");
 scanf("%d", & n);
 printf("\nEnter the cost adjacency matrix:\n");
 for (i = 1; i <= n; i++) {
  for (j = 1; j <= n; j++) {
   scanf("%d", & cost[i][j]);
   if (cost[i][j] == 0)
    cost[i][j] = 999;
  }
 }
 printf("The edges of Minimum Cost Spanning Tree are\n");
 while (ne < n) {
  for (i = 1, min = 999; i <= n; i++) {
   for (j = 1; j <= n; j++) {
    if (cost[i][j] < min) {
     min = cost[i][j];
     a = u = i;
     b = v = j;
    }
   }
  }
```

find(int i)

1. Start
2. Check whether parent[i] is not zero
    - If true, set i = parent[i] and repeat step 2
3. When parent[i] == 0, return i
4. Stop

uni(int i, int j)

1. Start
2. Check whether i != j
    - If true:
        - Set parent[j] = i
        - Return 1
3. Otherwise return 0
4. Stop

OUTPUT:

```
Implementation of Kruskal's algorithm
Enter the no. of vertices:5
Enter the cost adjacency matrix:
0   2   1   0   0
2   0   0   4   5
1   0   0   0   2
0   4   0   0   0
0   0   2   0   0
The edges of Minimum Cost Spanning Tree are
1 edge (1,3) =1
2 edge (1,2) =2
3 edge (3,5) =2
4 edge (2,4) =4
```

```
u = find(u);
v = find(v);
if (uni(u, v)) {
      printf("%d edge (%d,%d) =%d\n", ne++, a, b, min);
      mincost += min;
    }
    cost[a][b] = cost[b][a] = 999;
  }
 printf("\nMinimum cost = %d\n", mincost);
}
int find(int i) {
  while (parent[i])
    i = parent[i];
  return i;
}
int uni(int i, int j) {
  if (i != j) {
    parent[j] = i;
    return 1;
  }
  return 0;
}
```

RESULT:

Program is executed successfully and output is obtained.

ALGORITHM:

main()

1. Start
2. Display message “Enter the number of nodes:”
3. Read n (number of nodes)
4. Display message “Enter the adjacency matrix:”
5. For i = 1 to n
   For j = 1 to n
   - Read cost[i][j]
   - If cost[i][j] == 0, set cost[i][j] = 999
6. Mark the first node as visited:
   - Set visited[1] = 1
7. Repeat steps 9–17 while ne < n
8. Set min = 999
9. For i = 1 to n
   For j = 1 to n
   - Check if cost[i][j] < min
   - AND if visited[i] != 0
     - If true:
       - Set min = cost[i][j]
       - Set a = u = i
       - Set b = v = j
10. After scanning all edges, check if
    - visited[u] == 0 OR visited[v] == 0
11. If true:
    - Display the selected edge:
      “Edge ne : (a b) cost : min”
    - Increment ne by 1
    - Add min to mincost
    - Mark node b as visited (visited[b] = 1)
12. Set cost[a][b] = 999
13. Set cost[b][a] = 999
14. Continue loop until ne == n
15. After loop ends, display:
    “Minimum cost = mincost”
16. Stop

DATE: 19/11/2025

EXPERIMENT: 17

PRIM'S ALGORITHM

AIM:

Write a C program to implement prim's algorithm.

SOURCE CODE:

```
#include<stdio.h>

int a, b, u, v, n, i, j, ne = 1;
int visited[10] = {
 0
}, min, mincost = 0, cost[10][10];

void main() {
 printf("\nEnter the number of nodes:");
 scanf("%d", & n);
 printf("\nEnter the adjacency matrix:\n");

 for (i = 1; i <= n; i++)
  for (j = 1; j <= n; j++)
  {
   scanf("%d", & cost[i][j]);
   if (cost[i][j] == 0)
    cost[i][j] = 999;
  }

 visited[1] = 1;
 printf("\n");
 while (ne < n) {
  for (i = 1, min = 999; i <= n; i++)
   for (j = 1; j <= n; j++)
    if (cost[i][j] < min)
     if (visited[i] != 0) {
      min = cost[i][j];
      a = u = i;
      b = v = j;
     }

  if (visited[u] == 0 || visited[v] == 0)
  {
   printf("\n Edge %d:(%d %d) cost:%d", ne++, a, b, min);
```

OUTPUT:

Enter the number of nodes:5

Enter the adjacency matrix:

0	2	1	0	0
2	0	0	4	5
1	0	0	0	2
0	4	0	0	0
0	0	2	0	0

Edge 1:(1 3) cost:1

Edge 2:(1 2) cost:2

Edge 3:(3 5) cost:2

Edge 4:(2 4) cost:4

Minimum cost=9

```
    mincost += min;
    visited[b] = 1;
   }
   cost[a][b] = cost[b][a] = 999;
  }
  printf("\n Minimun cost=%d", mincost);
}
```

RESULT:

Program is executed successfully and output is obtained.